PORTFOLIO PROJECT · COMPLETE GUIDE

4×4 INT8 Systolic-Array Matrix-Multiply Accelerator

From one processing element to handwritten-digit recognition on a Basys3 FPGA. Every step, every design decision, every bug, every number, and the interview answers.

Verilog-2005 | Vivado 2022.1 | Basys3 · Artix-7 xc7a35tcpg236-1 | AXI4-Lite | UART → AXI bridge | Python / NumPy golden model | INT8 MNIST

16 DSP48E1, one per PE	**163 MHz** Fmax after timing fixes (WNS +3.866 ns)	**97.0%** INT8 MNIST accuracy, bit-exact to NumPy	**2.9×** speed-up from packed bus writes

Author: GJVS · Built step by step, October 2026 · All numbers in this guide come from your own Vivado 2022.1 runs, or from Icarus Verilog runs that your Vivado runs then reproduced exactly.

Contents

0. The project in one page
1. Background you must know
2. Vivado 2022.1 working flow (and its traps)
3. Step 1: the processing element (PE)
4. Step 2: the 4×4 array
5. Step 3: skew logic in hardware
6. Step 4: controller FSM
7. Step 5: AXI4-Lite wrapper
8. Step 6: system test vs Python/NumPy
9. Step 7: Basys3 build, DSPs and timing closure
10. Step 8: MNIST on the accelerator
11. Step 9a: packed writes (2.9× faster)
12. On the Basys3 board: bring-up and demos
13. Every problem we hit, and the fix
14. Numbers cheat sheet
15. Interview preparation: pitch, Q&A, resume
16. What comes next (9b, 10, ASIC)
A. Register map and formulas
B. File map: which file, which step, where in Vivado

How to use this guide

- **Before an interview:** read chapter 0, then chapters 14 and 15. That is about 20 minutes.
- **To understand any step again:** each step chapter has the same layout: *goal → concept → key code → design decisions → how to run in Vivado → results → bug proof → interview questions*.
- **When something in Vivado breaks:** chapter 13 lists every error we have already hit, with its fix.
- **On board day:** chapter 12 is a step-by-step checklist with the expected output for every command.

> **The one-sentence story**
> I designed a 4×4 INT8 output-stationary systolic array in Verilog. I wrapped it as an AXI4-Lite peripheral, verified it bit-exact against a NumPy golden model, and closed timing on a Basys3 at 163 MHz using 16 DSP48 slices. I then ran a real INT8 neural network (MNIST, 97% accuracy) on it through tiling, measured that the bus, not the compute, was the bottleneck, and made the system 2.9× faster with packed writes.

CHAPTER 0

The project in one page

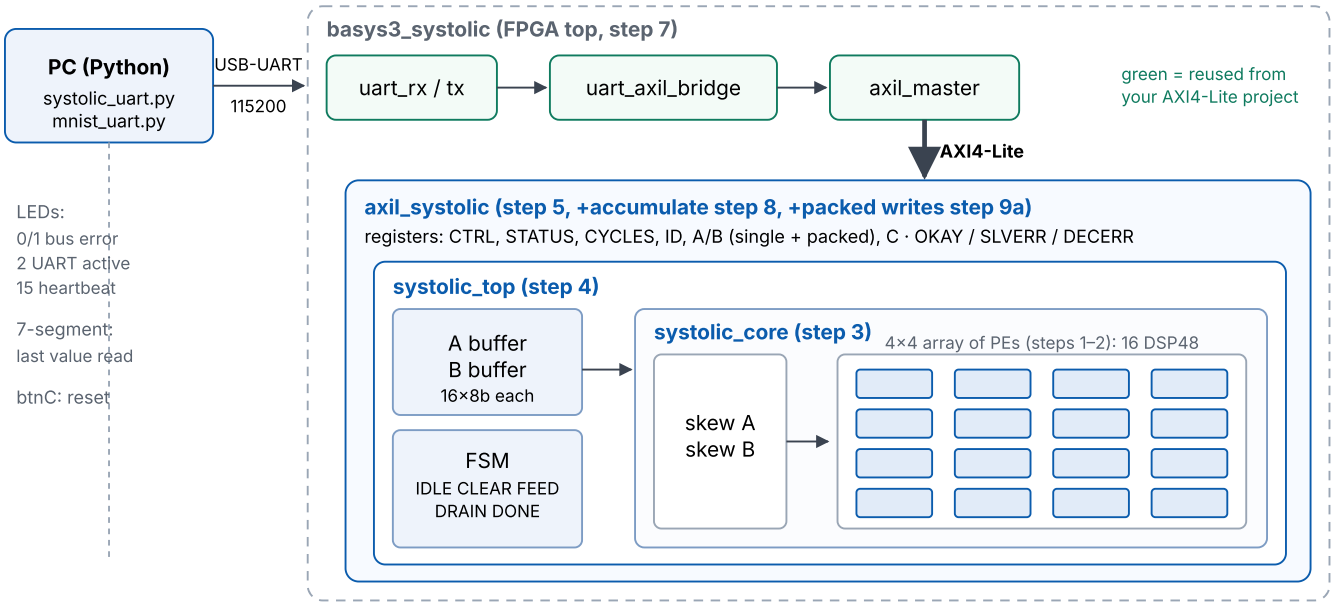


Figure 0.1: the complete system. Each box is one Verilog module, and each label says the step that built it.

11 clk
one 4×4 multiply, start→done (3N−1)

511 / 986
LUTs / flip-flops (≈2.5% of the chip)

2.6 GMAC/s
peak compute, 16 MAC × 163 MHz

16,000
results checked vs NumPy (1000 tests)

Step	What was built	Key result
1	Processing element: signed 8×8 multiply + 32-bit accumulate + pass registers	1000 random dot products pass
2	4×4 grid of PEs with `generate` loops	C correct; PE(i,j) final after clock i+j+4; also 8×8
3	Skew (triangle of shift registers) in hardware	Same C, same latency; broken skew caught (5009 errors)
4	Buffers + FSM (IDLE/CLEAR/FEED/DRAIN/DONE) + cycle counter	start→done = 11 clocks; short DRAIN caught (503 errors)
5	AXI4-Lite slave with register map and error responses	200 random jobs, 0 protocol violations; job = 249 clk vs 11 compute
6	System test vs NumPy golden model (1000 tests from .mem files)	16,000 results bit-exact; overflow bug found *only* by a corner case
7	Basys3 top: UART→AXI bridge, LEDs, 7-seg; DSP mapping and timing closure	0 → 16 DSPs; 125 → 146 → 173 MHz
8	Accumulate mode + tiling; INT8 MNIST MLP 196→32→10	97.00% INT8 (97.03% float); bit-exact; array busy only 6%
9a	Packed writes: 4 INT8 values per 32-bit bus write	19,000 → 6,520 clk/image (2.9×); 163 MHz
Board	Program Basys3, run ping/demo/test, then MNIST over UART	Chapter 12 checklist

CHAPTER 1

Background you must know

1.1 Matrix multiply is just dot products

C = A × B with 4×4 matrices: every element C[i][j] is the dot product of **row i of A** and **column j of B**:

```
C[i][j] = A[i][0]·B[0][j] + A[i][1]·B[1][j] + A[i][2]·B[2][j] + A[i][3]·B[3][j]
```

Each term is one **MAC** (multiply-accumulate). A 4×4×4 multiply is 4×4×4 = **64 MACs**. Neural networks are almost entirely matrix multiplies, which is why GPUs (tensor cores) and TPUs are built around MAC arrays.

1.2 Why a systolic array?

A CPU does one MAC at a time and fetches both operands from memory every time. A systolic array places a grid of MAC units (PEs) side by side, and lets data **flow through them like a pulse** (that is what "systolic" means, as in heartbeats):

- Each value is fetched from memory **once** and then reused by every PE it passes through. A value of A is used by 4 PEs in its row, and a value of B by 4 PEs in its column.
- Every wire is **short**: PE to neighbour PE only. So the clock speed does not drop when the array gets bigger.
- All 16 PEs work in parallel: up to 16 MACs every clock.

1.3 Output-stationary (ours) vs weight-stationary (TPU)

Dataflow	What stays in the PE	What moves	Used by
Output-stationary (this project)	the result C[i][j] (accumulator)	A moves right, B moves down	simple; good when K is long; results read out at the end
Weight-stationary	one weight B[k][j]	activations move right, partial sums move down	Google TPU v1 (256×256)
Input-stationary	one input A value	weights and partial sums move	less common

1.4 Skew: why the inputs must be staggered

A value entering row i moves one PE to the right per clock, so it reaches column j after j clocks. A value entering column j reaches row i after i clocks. For A[i][k] and B[k][j] to meet **in the same clock** at PE(i,j), row i must start i clocks late and column j must start j clocks late. That staircase is the "skew". Without it, the wrong pairs meet (step 2 proved it: 6012 wrong values).

1.5 INT8 and the accumulator width

- Signed INT8 range: −128 … 127. A product of two INT8 values needs 16 bits; the extreme is (−128)·(−128) = 16384.
- Summing K products needs 16 + $\log_2 K$ bits. For K = 4: worst case 4 × 16384 = 65536, which needs **18 bits** signed.

- We use **32-bit accumulators**. That allows K up to about 131,000 terms without overflow, which is enough for tiling large layers (step 8 uses K = 196).
- Why INT8 at all? 4× less memory and bandwidth than FP32, much smaller multipliers, and nearly the same accuracy (our MNIST: 97.03% float vs 97.00% INT8).

1.6 FPGA building blocks

Resource	What it is	In this design
LUT	6-input look-up table: any small logic function	decode, FSM, muxes (511)
FF (flip-flop)	1-bit register	pipeline, pass registers, AXI state (986)
LUTRAM	a LUT used as a tiny RAM	the A/B buffers (48 LUTs)
DSP48E1	hard multiplier (25×18) + 48-bit adder/accumulator	**16, one per PE**; Basys3 has 90
BRAM	36 Kb block RAM	0 (buffers are too small to need it)

1.7 AXI4-Lite in 60 seconds

AXI4-Lite is ARM's simple register bus, standard on Xilinx and Arm SoCs. It has **5 channels**, each with its own VALID/READY handshake: a transfer happens on the clock edge where both VALID and READY are 1.

Channel	Direction	Carries
AW (write address)	master → slave	awaddr
W (write data)	master → slave	wdata, wstrb (which bytes are valid)
B (write response)	slave → master	bresp
AR (read address)	master → slave	araddr
R (read data)	slave → master	rdata, rresp

Responses: **OKAY** (00), **SLVERR** (10: the slave refused the access), **DECERR** (11: no slave/register at that address). The key rules: once VALID is high it must stay high until READY; the payload must stay stable while VALID is waiting; READY must not depend on VALID combinationally in a way that deadlocks.

CHAPTER 2

Vivado 2022.1 working flow

Your project is `C:/Users/SHIVAKUMAR/verilog projects/systolic_array/`, part **xc7a35tcpg236-1** (Basys3). Every step used these same actions.

2.1 The three source sets

Set	Holds	Top in this project
Design Sources	real hardware (synthesised)	`basys3_systolic` (from step 7)
Simulation Sources	testbenches, .mem data files, test-only helpers	whichever `tb_*` you are running
Constraints	.xdc pins and clock	`basys3_systolic.xdc`

> **The rule that confused us once**
> `axil_master.v` started as a simulation helper (step 5), but from step 7 it is **real hardware** on the board, so it must be under Design Sources. A file that is only in Simulation Sources is not synthesised.

2.2 Adding or replacing a file

1. **Replace an existing file:** open it from the Sources window → Ctrl+A → paste the new version → Ctrl+S. Editing the file through the Sources tree guarantees you edit the copy Vivado actually uses.
2. **New file:** Add Sources → *Add or create design sources* (or *simulation sources*) → Create File (exact name) → **Finish** → paste → save. If you close the dialog without pressing Finish, nothing is added.
3. **Adding files from disk:** tick *Copy sources into project*. For .mem files set "Files of type" to **All Files**.

2.3 Running a simulation

1. Close any open simulation first (otherwise you get a `simulate.log` lock error).
2. Simulation Sources → right-click the testbench → **Set as Top**.
3. Set the run time once: Settings → Simulation → Simulation tab → `xsim.simulate.runtime` = **-all**. The default is 1000 ns, which stops long tests early. (Or type `run -all` in the Tcl console.)
4. Run Simulation → Run Behavioral Simulation. Read the Tcl console for `PASS`.
5. Warnings like *"Can't add object ... exceeds the display limit of 65536 bits"* are harmless: big memories just aren't drawn in the waveform.

2.4 .mem files: what they are

`$readmemh("a.mem", mem)` loads a text file of hex numbers (one per line) into a Verilog array at the start of the simulation. The .mem files are **data, not code**: Python (`golden.py`, `mnist_model.py`) generates them, and the testbench reads them. Vivado copies .mem files from Simulation Sources into the simulation folder automatically.

> **Notepad trap**
> Notepad saves `x.mem` as `x.mem.txt`. Fix: File Explorer → View → Show → **File name extensions**, then rename. Or in PowerShell in that folder: `Get-ChildItem *.mem.txt | Rename-Item -NewName { $_.Name -replace '\.txt$','' }`.

2.5 Synthesis → implementation → bitstream

1. **Run Synthesis** → OK. Then **Run Implementation** → OK (a few minutes).
2. Open Implemented Design → Reports → **Report Utilization** (LUT/FF/DSP/BRAM) and **Report Timing Summary** (WNS).
3. **Fmax = 1000 / (10 − WNS)** MHz, because the constraint is a 10 ns (100 MHz) clock. WNS must be positive.
4. Click the worst path to see its start and end: that is your **critical path**.
5. **Generate Bitstream** → `systolic_array.runs/impl_1/basys3_systolic.bit`.

> **Path-with-a-space trap (Common 17-165)**
>
> Because the project path contains a space ("verilog projects"), incremental synthesis fails on re-runs with `read_checkpoint` errors. Fix once in the Tcl console:
>
> ```
> set_property AUTO_INCREMENTAL_CHECKPOINT 0 [get_runs synth_1]
> set_property INCREMENTAL_CHECKPOINT {} [get_runs synth_1]
> reset_run synth_1
> ```
>
> and remove the `utils_1` .dcp from the project. Long term: keep projects in paths without spaces.

CHAPTER 3

1 The processing element (PE)

Goal: one MAC unit that also passes its inputs to its neighbours. The whole array is 16 copies of it, so if the PE is right, most of the design is right.

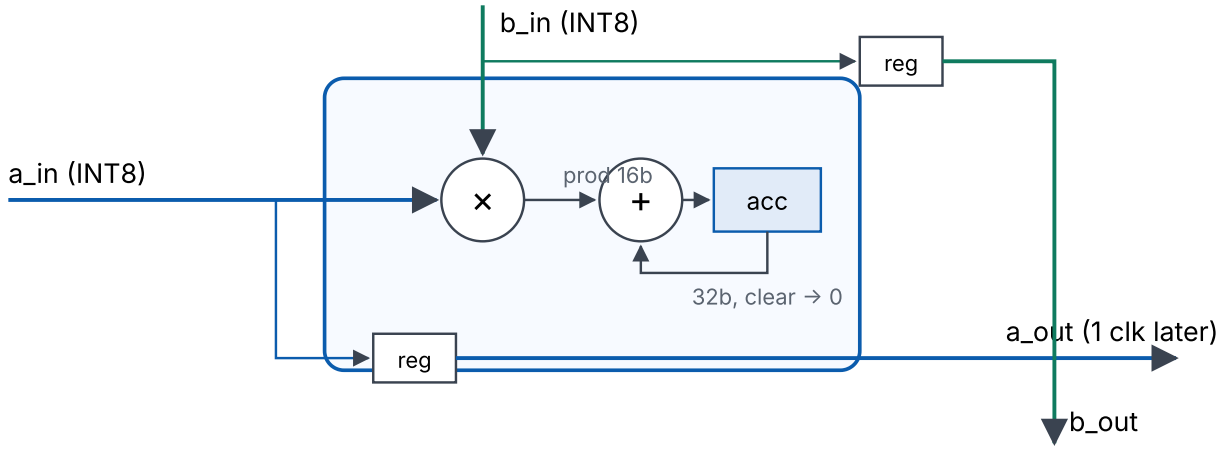


Figure 3.1: every clock the PE does acc += a_in × b_in, and hands a and b to its right and bottom neighbours one clock later.

Key code: `pe.v`

```
(* use_dsp = "yes" *)                      // added in step 7: map to a DSP48
module pe #( parameter DW = 8, parameter AW = 32 )(
    input  wire clk, rst, clear,
    input  wire signed [DW-1:0] a_in, b_in,
    output reg  signed [DW-1:0] a_out, b_out,
    (* use_dsp = "yes" *) output reg signed [AW-1:0] acc );

    (* use_dsp = "yes" *) wire signed [2*DW-1:0] prod = a_in * b_in;

    always @(posedge clk)
        if (rst) begin a_out <= 0; b_out <= 0; acc <= 0; end
        else begin
            a_out <= a_in;               // pass on: this delay makes it "systolic"
            b_out <= b_in;
            if (clear) acc <= 0;
            else       acc <= acc + prod; // prod is sign-extended to 32 bits
        end
endmodule
```

Design decisions

Choice	Why	What breaks otherwise
`signed` on every port and on `prod`	Verilog multiplies unsigned unless **both** operands are signed	−1 × −1 becomes 255 × 255 = 65025. Removing it: 999/1000 tests fail
16-bit product, 32-bit accumulator	8b×8b fits 16b; the sum needs 16+$\log_2 K$ bits	16-bit acc overflows at 4 × 16384 = 65536
Registered `a_out` / `b_out`	data moves one PE per clock; wires stay neighbour-to-neighbour	A would ripple through all PEs in one clock: long path, lower Fmax, wrong timing
`clear` input	zero acc for a new matrix without a full reset	old results add into the new ones
Synchronous active-high reset	one style across the design; maps onto FPGA flip-flops and DSP reset pins	(async is fine, but mixing styles causes trouble)

Verification (`tb_pe.v`)

```
Test 1 corner cases : acc = 129 (expected 129)     (-128)(-128) + 127(-128) + (-1)(-1) + 0
Test 2 pass-through : a_out=37 b_out=-90 one clock later
Test 3 random       : 1000 dot products checked
PASS: pe, 0 errors
```

Vivado: add `pe.v` (design) and `tb_pe.v` (simulation), set `tb_pe` as top, run. Verified by you on 2026-10-24.

Interview questions

What is a PE?
One MAC plus registers that pass the operands on to its neighbours.

Why register the outputs?
So data moves one hop per clock and wires stay local. Fmax then does not depend on the array size.

How does Verilog handle a signed multiply?
Both operands must be signed. If one is unsigned, the whole expression is unsigned. This is the most common INT8 MAC bug.

You changed pe.v once. How did that change all 16 PEs?
pe.v is a module (a template). The array instantiates it 16 times with a generate loop, so every instance is built from the same description.

CHAPTER 4

2 The 4×4 array

Goal: wire 16 PEs into a grid. No new logic, only wiring. A moves right, B moves down, and PE(i,j) ends up holding C[i][j].

```
           b_top[0]  b_top[1]  b_top[2]  b_top[3]
              |         |         |         |
  a_left[0] -> PE00 ---> PE01 ---> PE02 ---> PE03 -> (unused, removed by synthesis)
              |         |         |         |
  a_left[1] -> PE10 ---> PE11 ---> PE12 ---> PE13
              |         |         |         |
  a_left[2] -> PE20 ---> PE21 ---> PE22 ---> PE23
              |         |         |         |
  a_left[3] -> PE30 ---> PE31 ---> PE32 ---> PE33
```

Key code: `systolic_array.v`

```
wire [DW-1:0] a_h [0:N-1][0:N];     // a_h[i][j] = A value entering PE(i,j)
wire [DW-1:0] b_v [0:N][0:N-1];     // b_v[i][j] = B value entering PE(i,j)
genvar i, j;
generate
  for (i = 0; i < N; i = i + 1) begin : g_row
    for (j = 0; j < N; j = j + 1) begin : g_col
      pe #(.DW(DW), .AW(AW)) u_pe (
        .clk(clk), .rst(rst), .clear(clear),
        .a_in(a_h[i][j]),   .b_in(b_v[i][j]),
        .a_out(a_h[i][j+1]), .b_out(b_v[i+1][j]),      // the systolic wiring
        .acc(acc_flat[(i*N+j)*AW +: AW]));
    end
  end
endgenerate
```

Code idea	Why
nested `generate for` with `genvar`	builds N×N instances at elaboration time; change N and the grid rebuilds (8×8 also tested)
flattened ports, `x[i*DW +: DW]`	Verilog-2005 ports cannot be 2-D arrays; `+:` means "DW bits starting at this index"
no skew inside the array	in this step the testbench staggers the inputs; step 3 moves it into hardware

The walkthrough example (used everywhere afterwards)

```
A = [[1,2,0,-1],[3,1,2,0],[0,1,1,2],[2,0,1,1]]     C = A x B =   5  0  1  6
B = [[1,0,2,1],[2,1,0,3],[1,3,1,0],[0,2,1,1]]                     7  7  8  6
                                                                   3  8  3  5
                                                                   3  5  6  3
```

Results

- C matches NumPy; 500 random 4×4 and 500 random 8×8 multiplies pass.
- **Finish-time check:** PE(i,j) holds its final value after clock i+j+4; the last one, PE(3,3), finishes on clock 10 = **3N−2**.
- **Proof skew is needed:** removing the A skew gave 6012 wrong values and 252 timing errors.

Interview questions

Why genvar and not integer?

A generate loop builds hardware at elaboration time, so its variable must be a genvar. It does not exist in the final circuit.

What is the latency formula and why?

3N−2. Feeding K = N values takes N clocks, the skew delays the last row by N−1 clocks, and that row's last value must then travel N−1 PEs to the right to reach PE(N−1,N−1): N + (N−1) + (N−1) = 3N−2. That is 10 clocks for 4×4 and 22 for 8×8.

How large an array fits on the Basys3?

One DSP per PE and 90 DSPs, so 9×9 = 81 is the largest square array. Since N must be a power of two for our address decode, 8×8 (64 DSPs) is the practical maximum.

CHAPTER 5

3 Skew logic in hardware

Goal: the user feeds one column of A and one row of B per clock, all N values at once, and hardware creates the staircase.

```
lane 0 : in ----------------------> out   (wire, 0 clocks)
lane 1 : in --[R]-----------------> out   (1 clock)
lane 2 : in --[R]--[R]------------> out   (2 clocks)
lane 3 : in --[R]--[R]--[R]-------> out   (3 clocks)        0+1+2+3 = 6 registers per matrix
```

Clock	row 0	row 1	row 2	row 3
1	a00	a10	a20	a30
2	a01	a11	a21	a31
3	a02	a12	a22	a32
4	a03	a13	a23	a33
what you give (one column per clock)				

→

Clock	row 0	row 1	row 2	row 3
1	a00	0	0	0
2	a01	a10	0	0
3	a02	a11	a20	0
4	a03	a12	a21	a30
what the array gets (staircase; continues to clock 7)				

Key code: `skew.v`

```
assign out_flat[0 +: DW] = in_flat[0 +: DW];          // lane 0: a wire
generate for (i = 1; i < N; i = i + 1) begin : g_lane
    reg [DW-1:0] sr [0:i-1];                           // lane i has exactly i registers
    always @(posedge clk)
        if (rst) for (m = 0; m < i; m = m + 1) sr[m] <= 0;
        else begin
            sr[0] <= in_flat[i*DW +: DW];
            for (m = 1; m < i; m = m + 1) sr[m] <= sr[m-1];   // shift
        end
    assign out_flat[i*DW +: DW] = sr[i-1];
end endgenerate
```

`systolic_core.v` = skew for A + skew for B + the step 2 array.

Results

- `tb_skew`: random data for 1000 clocks, lane i delayed by exactly i. PASS.
- `tb_systolic_core`: same C, **same 10-clock latency** (lane 0 is a wire, so skew adds nothing). N=4 and N=8 pass.
- **Bug proof:** every lane delayed by only 1 clock → 5009 errors.
- Verified by you in Vivado on 2026-10-24 (steps 1–3).

Interview questions

How did you implement the skew and what does it cost?

A triangle of shift registers: lane i has i flip-flops, N(N−1)/2 registers per matrix. For 4×4 that is 6 × 8 bits per matrix, 96 FFs in total. It adds no latency.

Could memory produce skewed data instead?

Yes, by reading staggered addresses over time, but a shift register keeps the memory reads simple.

How did you verify it?

A unit test on the skew alone (delay = i on random data), then the full core against a reference, plus a deliberately broken skew to prove the testbench catches it.

CHAPTER 6

4 The controller FSM

Goal: a real accelerator interface: write A and B into buffers, pulse `start`, wait for `done`, read C. The FSM now does what the testbench did before.

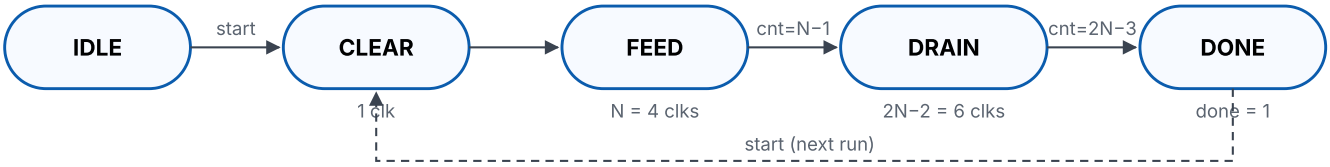


Figure 6.1: 1 + 4 + 6 = 11 clocks from start to done (3N−1). The array's 10 plus one CLEAR clock.

State	Clocks	Does	Why
IDLE	until start	waits; buffers writable	—
CLEAR	1	`clear` to all PEs (skipped in accumulate mode, step 8)	remove the old result
FEED	N	column cnt of A, row cnt of B	all K inputs, staggered by the skew
DRAIN	2N−2	zeros	data travels (N−1)+(N−1) PEs to PE(3,3)
DONE	until next start	`done` = 1, C readable	a level, so slow software polling can't miss it

Design decisions

Decision	Why
On-chip A/B buffers (16×8b each, LUT-RAM)	the host writes once; the FSM fetches column cnt itself
Writes and start ignored while busy	data cannot change mid-run; a double start can't restart a run
`done` is a level, not a pulse	polling over a bus could miss a 1-clock pulse
`busy` / `done` decoded from the state	no extra FFs, and they can never disagree with the FSM
`cycles` performance counter	hardware measures its own latency: you quote a measured 11, not a calculated number
Registered feed with 1-clock lookahead (added in step 7)	the DSP inputs come straight from FFs; this moved Fmax from 125 to 146 MHz (chapter 9)

Key code (final version, with the step 7 lookahead and step 8 accumulate)

```
wire          feed_next = (state == S_CLEAR) || (state == S_FEED && cnt != N-1);
wire [IW-1:0] k_next    = (state == S_CLEAR) ? 0 : cnt + 1;
always @(posedge clk)
  for (r = 0; r < N; r = r + 1)
    if (rst || !feed_next) begin a_col_flat[r*DW +: DW] <= 0; b_row_flat[r*DW +: DW] <= 0; end
    else begin
      a_col_flat[r*DW +: DW] <= a_buf[r*N + k_next];   // A[r][k]
      b_row_flat[r*DW +: DW] <= b_buf[k_next*N + r];   // B[k][r]
    end
// core: .clear(state == S_CLEAR && !acc_mode)
```

Results

```
Test 1 walkthrough example: start -> done in 11 clocks (expected 11)
Test 2 writes/start while busy ignored: ok
Test 3 random: 500 signed 4x4 runs, back to back, every run 11 clocks
PASS: systolic_top N=4, 0 errors          (8x8: 500 runs, 23 clocks each)
```

Bug proof: allowing writes while busy → 4 errors; DRAIN one clock short → 503 errors. Verified in Vivado (finish at 391,756 ns).

Interview questions

Why is DRAIN 2N−2 clocks?

The last inputs must travel N−1 PEs right and N−1 PEs down to reach PE(N−1,N−1). One clock short gave 503 errors in the far corner.

What is the measured latency?

11 clocks for 4×4 (1 clear + 4 feed + 6 drain), which is 110 ns at 100 MHz and 67 ns at 163 MHz.

How would you raise throughput?

Overlap runs: double-buffer A/B so the next tile loads while the current one computes, and read C while the next run starts.

CHAPTER 7

5 The AXI4-Lite wrapper

Goal: make the accelerator a standard bus peripheral, so a CPU (MicroBlaze, Zynq ARM) or your UART→AXI bridge can drive it with plain register reads and writes. The handshake style is reused from your `axil_mem_slave.v`, and the testbench reuses your `axil_master.v` and `axil_checker.v` unchanged.

Register map

Address	Name	Access	Meaning
0x000	CTRL	W	bit0 = start (self-clearing); bit1 = accumulate (step 8)
0x004	STATUS	R	bit0 = busy, bit1 = done
0x008	CYCLES	R	clocks of the last run (11)
0x00C	ID	R	0x5A000408 = "SA", N=4, 8-bit; software checks it first
0x100 + 4w / 0x200 + 4w	A / B packed	W	step 9a: 4 INT8 values per write
0x400 + 4(iN+j)	A[i][j]	W	low byte = signed INT8 (e.g. A[1][2] = 0x418)
0x800 + 4(iN+j)	B[i][j]	W	low byte = signed INT8
0xC00 + 4(iN+j)	C[i][j]	R	signed 32-bit result (C[3][3] = 0xC3C)

Error responses

Situation	Response
normal access	OKAY
misaligned address (not a multiple of 4)	SLVERR
wrong direction: read A/B, write C/STATUS/CYCLES/ID	SLVERR
write A, B or CTRL while busy	SLVERR (refuse loudly instead of silently corrupting)
no register at that address	DECERR

Write path structure (the important handshake idea)

```
assign awready = !aw_full;            // READY never depends on VALID
assign wready  = !w_full;
// AW and W are latched independently (they may arrive in any order)
wire do_write = aw_full && w_full && (!bvalid || bready) && !pk_busy;  // pk_busy: step 9a
// on do_write: decode region(aw_addr_q), act if OKAY, then raise bvalid with bresp
```

Row and column come straight from address bits (`a[9:2] >> log2 N`, `a[9:2] & (N-1)`), so no divider is needed. That is why N must be a power of two.

Results

```
Test 1 ID register = 0x5a000408 ; C read over AXI = walkthrough matrix
   one full job over the bus: 249 clocks (compute itself: 11 clocks)
Test 2 error responses (SLVERR/DECERR): ok
Test 3 writes while busy refused, data intact: ok
Test 4 random: 200 signed 4x4 jobs over AXI4-Lite
AXI protocol checker: 0 violations
PASS: axil_systolic N=4, 0 errors          (8x8 also passes)
```

> **The most important number: 249 vs 11**
> The math takes 11 clocks, but the job over AXI4-Lite takes 249: **about 95% of the time is data movement** (32 writes, polling, 16 reads). Compute is cheap and data movement is expensive. This lesson drives steps 8 and 9.

Verified by you in Vivado (finish 505,056 ns).

Interview questions

Why AXI4-Lite?
It is the standard control bus on Xilinx and Arm systems. Any CPU or bridge can use the block without custom wiring.

What happens if software writes A during a run?
SLVERR, and the data is unchanged. I tested it, including a re-run that proves A was not modified.

How do you know your AXI is correct?
A protocol checker watches every clock: VALID held until READY, stable payload, response ordering, no X on the bus, no EXOKAY. It found 0 violations over 200+ jobs.

SLVERR vs DECERR?
DECERR: no register exists at that address. SLVERR: the register exists, but this access is not allowed (wrong direction, misaligned, or busy).

CHAPTER 8

6 System test against Python/NumPy

Goal: an **independent golden model**. A Verilog reference written by the same person can share the same misunderstanding as the design. NumPy computes C = A @ B, and the RTL must match it through the real AXI bus.

```
golden.py gen  (NumPy)  -> meta.mem  a.mem  b.mem  c_expected.mem
                                  | $readmemh
tb_system -> axil_master -AXI4-Lite-> axil_systolic      (+ axil_checker on the bus)
    |-- compares every C with c_expected.mem            (check 1, in the simulator)
    '-- writes hw_results.txt -> golden.py check        (check 2, in Python)
```

The testbench contains **no math**. It only moves data and compares.

The 1000 test vectors: 9 directed corner cases + 991 random

#	Case	Why
0	walkthrough example	known answer
1	all zeros	clear/reset paths
2	identity × B	C must equal B
3	**−128 × −128**	**largest result, 65536, needs 18 bits**
4	−128 × 127	most negative, −65024
5	127 × 127	large positive
6	−1 × −1	sign extension: 0xFF is −1
7	checkerboard 127 / −128	mixed signs
8	only A[3][3] = 100	data must reach the farthest PE (DRAIN length)

Results

```
Tests run         : 1000 (0 failed)        Results compared : 16000
Bus clocks / job  : 245   (compute only: 11)
At 100 MHz        : 408163 jobs/s, 26 MMAC/s over AXI4-Lite
AXI protocol check: 0 violations
PASS: system test N=4, hardware == NumPy on all 1000 tests     (finish 2,450,076 ns)
N=8: 300 tests, 19,200 results, 929 bus clocks per job, 55 MMAC/s
```

Bug proof: why directed tests matter

Bug planted in pe.v	Tests failed (of 1000)	Caught by
unsigned multiply	997	almost everything
accumulator cut to 17 bits	**1**	**only test 3 (−128 × −128)**

> **The lesson to tell interviewers**
> 991 random tests missed the overflow bug, because random values almost never make all four products large with the same sign. Only the hand-picked corner case caught it. Real verification plans combine constrained-random tests with directed corner cases and coverage.

Interview questions

How did you verify the design?

Unit testbenches for every block, then a system test in which a NumPy golden model generated 1000 tests (16,000 results). The RTL had to match through the real AXI bus with a protocol checker running, and I planted bugs to prove the tests catch them.

Is random testing enough?

No. My accumulator-overflow bug failed only 1 of 1000 tests, and that one was the directed corner case.

What is the throughput?

64 MACs per 11 clocks of compute, but 245 clocks per job over AXI4-Lite, so 26 MMAC/s. The bus is the bottleneck.

CHAPTER 9

7 Basys3 build: DSPs and timing closure

Goal: answer the three questions every hardware interviewer asks. **Does it fit? How fast does it run? Does it work on silicon?** This step joins your two projects: the UART bridge and AXI master from the AXI4-Lite project now drive the accelerator.

The board top: `basys3_systolic.v`

Part	Detail
Modules	uart_rx, uart_tx, uart_axil_bridge, axil_master (reused) + axil_systolic + sevenseg_hex4
Clock	100 MHz on pin W5, `create_clock -period 10.000`
UART	RsRx = B18, RsTx = A18, 115200 baud (CLKS_PER_BIT = 868), over the same micro-USB cable used for programming
Reset	btnC (U18), 2-flop synchroniser + power-on counter, then a **registered reset with `(* max_fanout = 50 *)`**
LEDs	LED0 last write error · LED1 last read error · LED2 PC talking · LED15 heartbeat ≈1.5 Hz
7-segment	low 16 bits of the last value read (after `demo`: 0003 = C[3][3])
XDC	pins from Digilent's Basys-3-Master.xdc; `set_false_path` on button, UART and display I/O (human-speed signals); CONFIG_VOLTAGE 3.3, CFGBVS VCCO

UART frames (from your bridge): write = `'W'` + addr(4 bytes, big-endian) + data(4) + strb(1) → 1 status byte; read = `'R'` + addr(4) → status + data(4); ping → 0x50.

Timing closure, one fix at a time

Version	DSP	LUT	FF	WNS	Fmax	Critical path / problem
1. Default synthesis	0	2012	1411	—	—	8×8 multipliers built from LUTs
2. `use_dsp` in pe.v	16	549	771	+1.986	125 MHz	FSM cnt → buffer LUT-RAM (fanout 69) → feed mux → DSP of PE(0,0)
3. Registered lookahead feed	16			+3.142	146 MHz	reset LUT, fanout 714 → DSP reset pins
4. Registered, replicated reset	16			+4.233	**173 MHz**	final for step 7
5. + packed writes (step 9a)	16	511	986	+3.866	**163 MHz**	current design

Fmax = 1000 / (10 − WNS). Every version passed the same testbenches with the same 11-clock latency.

Fix 1: DSP mapping

Vivado mapped the small 8×8 multipliers to LUTs (0 DSPs, 2012 LUTs). Adding `(* use_dsp = "yes" *)` on the module, on the `prod` wire and on `acc` put each multiply-accumulate into one DSP48E1: **16 DSPs, LUTs −73%**. The DSP48E1 has a 25×18 multiplier followed by a 48-bit adder with feedback, which is exactly a MAC.

Fix 2: registered lookahead feed (+21% Fmax)

The slowest path went from the FSM counter, through the buffer's LUT-RAM read and the feed mux, into the DSP of PE(0,0). PE(0,0) has no skew register in front of it (lane 0 is a wire). Fix: read column k+1 one clock early into a register (`feed_next` , `k_next`), so the DSP inputs come straight from flip-flops. Latency is unchanged at 11.

Fix 3: reset fanout (+18% Fmax)

One LUT drove reset to about 714 loads, including all 16 DSP reset pins. Fix: a registered reset with `(* max_fanout = 50 *)` , so Vivado replicates the register and each copy drives at most 50 loads.

> **Peak compute**
> 16 MACs per clock × 173 MHz ≈ **2.8 GMAC/s** (2.6 GMAC/s at 163 MHz in the current design).

Board-level simulation (`tb_basys3_systolic.v`)

```
Ping reply: 0x50   (ok)          ID over UART: 0x5a000408
  test  0 ok   (7-seg shows 0003 = C[3][3]) ... test 9 ok (7-seg shows 0df7)
PASS: board design over UART, 10 jobs == NumPy          (finish 8,694,915 ns)
```

The testbench acts as the PC and talks only through the UART pin (CPB=16 to keep the simulation short).

Where the time goes

Path	Time per 4×4 job
compute inside the array	11 clocks = 110 ns
over AXI4-Lite (step 6)	245 clocks = 2.45 µs
over UART from the PC	≈ 50 ms (10 to 20 jobs/s)

UART is about 450,000× slower than the compute. This is the "memory wall" in miniature.

Interview questions

How many resources does it use?
16 DSP48E1 (one per PE), 511 LUTs, 986 FFs and 0 BRAM on an Artix-7 35T. That is about 2.5% of the logic and 18% of the DSPs.

What is your Fmax and what limited it?
163 MHz (WNS +3.866 ns at a 100 MHz constraint). Originally 125 MHz: the feed path from the counter through the LUT-RAM into the first DSP. A registered lookahead feed gave 146 MHz. Then reset fanout (714 loads) limited it, and a replicated registered reset gave 173 MHz. The packed-write logic in step 9a cost 6% (163 MHz).

Why did Vivado use 0 DSPs at first?
For small multipliers (8×8), Vivado's heuristics may prefer LUTs. The `use_dsp` attribute forces DSP mapping, including the accumulator adder.

Why are the I/O paths false paths?
The button and UART RX go through 2-flop synchronisers, and the LEDs and display are read by eye. These are asynchronous human-speed signals, so timing them against the 100 MHz clock is meaningless.

CHAPTER 10

8 MNIST on the accelerator

Goal: run a real neural network. Read handwritten digits from MNIST and classify them, with every matrix multiply done on the systolic array.

```
28x28 image --2x2 average--> 14x14 = 196 pixels (INT8, 0..127)
    |  layer 1:  [4 images x 196] x W1[196 x 32]   <- systolic array (392 runs)
    |  bias + ReLU + requantize to INT8           <- CPU / PC / testbench
    |  layer 2:  [4 x 32] x W2[32 x 12]            <- systolic array (24 runs)
    v
12 scores (10 digits + 2 padding columns with bias -2^30 that never win) -> argmax = digit
```

	Float32 (Python)	INT8 (what the FPGA computes)
Accuracy on all 10,000 MNIST test images	97.03%	**97.00%**

Idea 1: tiling with accumulate mode

```
C tile (4x4) = A(:, 0:3)     x W(0:3, :)        <- run 1:  CTRL = 1  (clear, then compute)
             + A(:, 4:7)     x W(4:7, :)        <- run 2:  CTRL = 3  (start + accumulate: C += A x
B)
             + ...
             + A(:, 192:195) x W(192:195, :)    <- run 49: CTRL = 3
```

- K = 196 is split into 49 chunks; the 16 accumulators sum them because **CTRL bit 1 = accumulate** skips the CLEAR.
- Output width 32 = 8 column tiles. Layer 1 = 49 × 8 = **392 runs**; layer 2 = 8 × 3 = **24 runs**; **416 runs** per batch of 4 images (104 per image).
- **The only hardware change:** `acc_mode` latched at start, and `.clear(state == S_CLEAR && !acc_mode)`. Everything else is software, which is how real accelerators work: the hardware does one fixed tile size, and the driver or compiler tiles any matrix onto it.

Idea 2: integer-only quantization

```
x_q  = round(pixel / 255 x 127)                          INT8 inputs, 0..127
acc1 = x_q . W1_q + b1_q                                 INT32 (accelerator)
h_q  = clamp( (max(acc1,0) x 40422 + 2^23) >> 24, 0, 127 )  ReLU + requantize (CPU)
acc2 = h_q . W2_q + b2_q                                 INT32 (accelerator)
digit = argmax(acc2)
```

Multiplying by M = 40422 and shifting right by S = 24 is the same as multiplying by about 0.0024, with no floating point. Weight scale = max|W|/127, and the hidden scale comes from the 99.9th percentile of real activations. Because every step is integer math, Python, the Verilog testbench and the FPGA give **bit-identical** scores.

Results (Icarus, then reproduced exactly in your Vivado)

```
Digits correct     : 19 / 20          (image 8 is a 5 the network calls a 6; Python agrees)
Scores vs NumPy    : 0 mismatches (bit-exact check of all 240 scores)
Accelerator runs   : 2080 (416 per batch of 4 images)
Clocks per image   : 19000 over AXI4-Lite
Array busy         : 22880 of 380001 clocks = 6.0%
At 100 MHz         : 5263 images/s
AXI protocol check : 0 violations
```

Bug proof: with accumulate mode broken (C cleared every run), only 2/20 digits are correct and 199/240 scores mismatch.

> **The key number: the array is busy only 6% of the time**
> Per image, the array computes for 104 runs × 11 = 1,144 clocks. The other ≈17,850 clocks are bus traffic: 32 writes per run, polling, and reading C. Fixes: packed writes (step 9a), double buffering (9b), weight reuse.

Files

`mnist_model.py` trains a NumPy MLP (hidden 32, 12 epochs, SGD with momentum, lr 0.1 × 0.7 per epoch), quantizes it, and writes `vectors/*.mem` (meta, x, w1, b1, w2, b2, scores, labels) plus `mnist_int8.npz`. `tb_mnist.v` plays the CPU: tiling, bias/ReLU/requantize and argmax, comparing every score. `mnist_uart.py` runs the same thing from the PC over UART, or in `sim` mode without a board.

Interview questions

How do you run a 196-wide layer on a 4×4 array?
Tiling. 4×4 output tiles, with K split into 49 chunks summed in the accumulators by an accumulate mode. That is 416 array runs per batch of 4 images.

How do you go from INT32 back to INT8?
Integer requantization: ReLU, multiply by M, add a rounding constant, shift right by S, clamp to 0..127. No floating point, so it is bit-exact to the Python model.

How much accuracy did INT8 cost?
0.03% (97.03% to 97.00% on 10,000 images). This is why inference hardware (tensor cores, TPUs) uses INT8.

Why is image 8 wrong if it is "bit-exact"?
The network itself gets it wrong, and Python agrees. Bit-exact means the hardware matches the model, mistakes included.

CHAPTER 11

9a Packed writes: 2.9× faster

Goal: attack the biggest piece of the bus time first. In step 8, every write carried one 8-bit value in a 32-bit word, so 3/4 of each write was empty.

```
before: write 0x400 = 0x000000_12   write 0x404 = 0x000000_F3   ...   (32 writes per tile)
after : write 0x100 = 0x2A_07_F3_12   byte k = element 4w+k (row-major)  (8 writes per tile)
```

Hardware: a 4-lane sequencer in `axil_systolic.v`

The A/B buffer has one write port, so a packed write is latched in `pk_data` and written out one byte per clock. **While `pk_busy` = 1, the next bus write waits** (`do_write && !pk_busy`), so a CTRL start can never overtake the data. AXI back-pressure does the waiting: READY simply stays low a little longer.

```
clock:        0        1        2        3        4
bus write  [0x100] ------------------------------ next write may commit
sequencer   latch   lane 0   lane 1   lane 2   lane 3
buffer              A[0][0]  A[0][1]  A[0][2]  A[0][3]     (lanes with WSTRB = 0 are skipped)
```

- New regions: A_PACK = 0x100 + 4w, B_PACK = 0x200 + 4w, with w < N·N/4. The old addresses still work, and every old testbench passes.
- WSTRB is honoured per byte, which is the AXI rule for partial writes.
- Reading a packed address gives SLVERR, past the end gives DECERR, a misaligned address gives SLVERR, and writing while busy gives SLVERR.
- Cost: about 42 FFs (data 32, strobe 4, lane 2, word index 2, busy, select) and a few LUTs.

Results (Icarus, then reproduced exactly in your Vivado)

	Step 8	Step 9a
Data writes per tile	32	**8**
Bus writes per image	3,432	**936**
Clocks per image	19,000	**6,520**
Images/s at 100 MHz	5,263	**15,337**
Array busy	6.0%	**17.5%**
Fmax (Vivado)	173 MHz	163 MHz

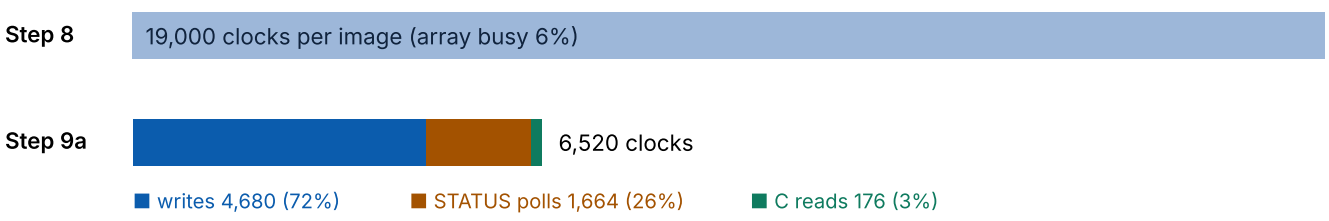


Figure 11.1: time per image, drawn to scale. From the two runs: a write costs 5 clocks and a read 4 clocks.

Why 2.9× and not 4×? Amdahl's law

Packing only shrinks the data writes. The CTRL write, the STATUS polling and the C reads are untouched. The polling (26%) is now the clear next target: the CPU sits waiting while the array computes. **Double**

buffering (9b) lets it load the next tile during the current run.

Verification: `tb_packed.v` (directed) + `tb_mnist_fast.v`

```
ok 1. packed A and B (3 random tiles, incl. -128 x -128)   ok 4. misaligned packed write -> SLVERR
ok 2. WSTRB partial write                                  ok 4. packed write while busy -> SLVERR
ok 3. mixed old + packed                                   ok 4. data unchanged by errors
ok 4. read A/B packed -> SLVERR                            ok 5. CTRL right after packed
ok 4. past the end -> DECERR                               ok 6. full-speed packed burst
AXI protocol check : 0 violations      PASS: packed writes     (Vivado: 9,566 ns)
tb_mnist_fast: 6520 clocks/image, 0 mismatches, PASS     (Vivado: 1,304,036 ns)
```

Bug planted	Caught by
Remove `!pk_busy` (no stall)	**only test 6** (16 errors)
Ignore WSTRB	tests 2, 3, 4
Bytes in reverse order	every test (112 errors)

> **Interview story: the bug the normal master could not see**
> `axil_master` takes 5 clocks per write, slower than the 4-clock sequencer, so removing the stall never failed with it. Test 6 takes over the bus with a raw driver that holds VALID high and sends a write every 2 clocks. Only that test catches the missing back-pressure. Lesson: test at the protocol's maximum rate, not just with your own master's timing.

Regression: tb_systolic_top, tb_axil_systolic (N=4 and N=8), tb_system (1000 tests), tb_basys3_systolic and tb_mnist (old addresses) all pass with the new slave.

Interview questions

Your utilization was 6%. What did you do?

I measured where the time went, then fixed the biggest piece first. Data writes were most of the traffic and 3/4 of each write was empty, so I packed 4 INT8 values per write: 2.9× faster, bit-exact, for about 42 flip-flops.

The buffer has one write port. How do 4 bytes get in?

A 4-lane sequencer writes one byte per clock and back-pressures the bus through READY while it is busy, so ordering is preserved.

Why did Fmax drop from 173 to 163 MHz?

Probably the new 4-to-1 byte select and the extra write-path control in front of the buffer. Trading 6% of clock speed for 2.9× more throughput is worth it. Pipelining the lane mux would recover the clock speed if needed.

CHAPTER 12

On the Basys3 board: bring-up and demos

Everything up to now ran in simulation and in Vivado's reports. This chapter turns it into a working board demo. It is a checklist: do the steps in order and compare against the expected output. **One bitstream covers every demo:** the current project already contains the step 9a slave, which still accepts the old one-element addresses.

12.1 What you need

Item	Where it comes from
Basys3 board + micro-USB cable	the cable gives power, JTAG programming *and* the UART
`basys3_systolic.bit`	`systolic_array.runs/impl_1/` after Generate Bitstream
`systolic_uart.py`	step 7 files: ping / regs / demo / test
`mnist_uart.py` (step 9 version) + `mnist_int8.npz`	step 9 page + step 8 folder in the Library
Python 3 with `numpy` and `pyserial`	`pip install numpy pyserial` (once)

Put the three Python-side files in one folder, for example `C:\systolic_demo\`. Open a command prompt there by clicking the File Explorer address bar, typing `cmd` and pressing Enter.

12.2 Before power-on

1. **Mode jumper JP1** (top-left, near the power switch): **JTAG** for programming from Vivado.
2. **Power jumper JP2**: **USB**.
3. Connect the micro-USB cable to the PROG/UART port and turn the **power switch ON**. The red power LED lights.

12.3 Program the FPGA

1. In Vivado: Flow Navigator → **Open Hardware Manager** → **Open Target** → **Auto Connect**.
2. The Hardware window shows `xc7a35t_0`. Right-click it → **Program Device**.
3. The bitstream field should already point to `…/impl_1/basys3_systolic.bit`. Click **Program**.
4. The green DONE LED on the board lights when configuration succeeds.

12.4 First signs of life (no PC software yet)

You should see	Meaning	If not
LED15 blinking ≈1.5 times per second	clock running, design loaded	re-program; check the DONE LED
7-segment shows `0000`	nothing read yet	check the `an` / `seg` pins in the XDC
LED0 and LED1 off	no bus errors	press btnC to reset

12.5 Find the COM port, and make the UART fast

1. Windows Device Manager → **Ports (COM & LPT)** → "USB Serial Port (COMx)". If two appear, it is usually the higher number. The examples use COM5; replace it with yours.
2. **Recommended:** right-click the port → Properties → Port Settings → Advanced → **Latency Timer = 1 ms** (the default is 16 ms). Every register access waits for a reply, so this speeds the demos up a lot.

12.6 Demo 1: matrix multiply (`systolic_uart.py`)

```
python systolic_uart.py COM5 ping
    ping: ok
    ID  : 0x5a000408                      <- "SA", 4x4, INT8: the board answers

python systolic_uart.py COM5 regs
    STATUS 0x00000000  (busy=0 done=0)    CYCLES 0    ID 0x5a000408

python systolic_uart.py COM5 demo
    Found 4x4 INT8 systolic array
    C from the FPGA = [[5 0 1 6] [7 7 8 6] [3 8 3 5] [3 5 6 3]]
    NumPy says      = (same)
    compute took 11 clocks on the FPGA
    PASS                                  <- 7-segment now shows 0003 (= C[3][3])

python systolic_uart.py COM5 test -n 200
    PASS: FPGA == NumPy on all 200 jobs   <- LED2 flickers during the run; LED0/LED1 stay off
```

12.7 Demo 2: handwritten digits (`mnist_uart.py`)

```
python mnist_uart.py sim -n 8             (no board: checks the script and the .npz)
python mnist_uart.py COM5 -n 8            (on the FPGA, packed writes)
      |   ++.        |
      |   #######+   |
      |        .#.   |    ...  ASCII art of each digit
      image 0: FPGA says 7 (label 7)
      ...
    8 / 8 correct, X.XX s per image.
python mnist_uart.py COM5 -n 8 --unpacked (step 8 style, one value per write)
```

- The first 8 test images are all classified correctly (image 8, the known mistake, is the 9th).
- **Measure both runs and write down the seconds per image.** The packed run should be roughly 2.5 to 3× faster over UART as well, because it sends about 1,400 bus transactions per image instead of about 3,900. That is a real hardware measurement of the step 9a improvement, worth quoting.
- Expect seconds per image, not microseconds: the array computes for about 11 µs per image, and everything else is the 115200-baud UART and USB round trips.

12.8 Optional: make the design survive power-off (flash)

1. Settings → Bitstream → tick **-bin_file**, then Generate Bitstream again.
2. Hardware Manager → right-click `xc7a35t_0` → **Add Configuration Memory Device** → choose the flash for your board revision (most Basys3 boards: `s25fl032p-spi-x1_x2_x4`; check Digilent's Basys3 reference manual if yours differs) → program it with the .bin.
3. Power off, set **JP1 to QSPI**, power on. The design now loads from flash by itself.

12.9 Troubleshooting

Symptom	Likely cause → fix
Hardware Manager finds no target	cable/driver. Try another cable or USB port (some cables are charge-only). Re-install the Digilent cable drivers: run `install_drivers_wrapper.bat` as Administrator from `C:\Xilinx\Vivado\2022.1\data\xicom\cable_drivers\nt64\`, or use the Vivado installer's cable-driver option.
LED15 not blinking after programming	wrong part or XDC not in the project; check the clock pin W5 and that the .xdc is under Constraints
`UART timeout` / `NO REPLY`	wrong COM port; port held open by another program (close serial terminals); press btnC; re-program
`could not open port` / access denied	another program (or a second Python window) has the port open
`ID register 0x…: this is not the systolic array`	an older bitstream from another project is loaded; re-program
`write 0x100 refused` in mnist_uart.py	bitstream built before step 9a; re-run synthesis and implementation and re-program, or use `--unpacked`
LED0 / LED1 on	a write or read got SLVERR/DECERR (e.g. wrong address); the scripts also print the failing address
Very slow MNIST demo	set the FTDI Latency Timer to 1 ms (12.5)

12.10 Capture it for your portfolio

- Record a 30–60 s phone video: the board with LED15 blinking, the terminal running `demo` (PASS, 7-seg shows 0003), then `mnist_uart.py` printing digits.
- Take screenshots of the utilization report, the timing summary (WNS +3.866 ns) and the implemented device view.
- Fill in this table and put it in the GitHub README:

Board result	Value
ping / ID	0x5A000408
demo (walkthrough C, 11 clocks)	PASS / FAIL
random test, 200 jobs vs NumPy	PASS / FAIL
MNIST first 8 images	__ / 8 correct
MNIST seconds per image, packed vs unpacked	____ s vs ____ s (speed-up ____ ×)

CHAPTER 13

Every problem we hit, and the fix

Interviewers love "tell me about a bug you found." These are all real, from this project.

#	Problem	Root cause	Fix / lesson
1	DSPs = 0, 2012 LUTs after synthesis	Vivado chose LUT multipliers for 8×8	`(* use_dsp = "yes" *)` on module, `prod` and `acc` → 16 DSPs, −73% LUTs
2	use_dsp edit "had no effect" the first time	the edited copy was not the file the project used (or was not saved)	edit through the Vivado Sources tree; check the path in Source File Properties
3	Common 17-165 `read_checkpoint` error on re-synthesis	space in "verilog projects" breaks incremental checkpoint	disable AUTO_INCREMENTAL_CHECKPOINT, clear INCREMENTAL_CHECKPOINT, remove utils_1 dcp, `reset_run synth_1`
4	Step 7 files missing from the project	Add Sources dialog closed without pressing Finish	re-add them and always press Finish
5	axil_master not synthesised	it was only in Simulation Sources	move to Design Sources once it becomes real hardware
6	Hierarchical reference `u_acc.acc_busy` in the board top	hierarchical references are not synthesizable	drive LEDs from real ports (bus error flags) instead
7	WNS +1.986 ns (125 MHz)	counter → LUT-RAM → mux → DSP of PE(0,0), which has no skew register	registered lookahead feed → 146 MHz, latency unchanged
8	WNS +3.142 ns	one reset LUT driving 714 loads incl. DSP reset pins	registered reset with `max_fanout = 50` → 173 MHz
9	.mem files not found	Notepad saved them as `.mem.txt`	show file extensions, rename (or the PowerShell one-liner)
10	Simulation "ends" at 1 µs	default xsim runtime is 1000 ns	`xsim.simulate.runtime = -all`
11	`simulate.log` locked	old simulation still open	close the simulation before switching the top
12	Accumulator overflow passed 999 of 1000 tests	random data never reaches the extremes	directed corner case −128 × −128 catches it
13	Missing bus stall invisible in normal tests	our master is slower than the sequencer	full-speed raw-bus test (step 9a test 6)
14	"Can't add object to the wave window" warnings	memories larger than 65,536 bits	harmless; ignore
15	Yosys "latch" warning on a loop integer	tool message about a `for` -loop variable	harmless; Vivado reports no latches (Register as Latch = 0)

CHAPTER 14

Numbers cheat sheet

Memorise this page. Every number comes from a run of this project.

Topic	Number	Where it comes from
Array	4×4, signed INT8 in, 32-bit accumulators, output-stationary	design
Array latency	3N−2 = 10 clocks; with CLEAR 3N−1 = **11** (8×8: 23)	steps 2 and 4, cycles counter
Work per run	64 MACs (4×4×4)	—
Worst-case sum	4 × 16384 = 65536 → 18 bits needed	step 6, test 3
Resources (current)	**16 DSP48E1, 511 LUT (48 as LUT-RAM), 986 FF, 0 BRAM**	your Vivado, placed design
Before use_dsp	0 DSP, 2012 LUT, 1411 FF	your Vivado
Fmax progression	125 → 146 → 173 MHz; **163 MHz** with packed writes	WNS 1.986 / 3.142 / 4.233 / 3.866 ns
Peak compute	16 MAC/clock → 2.6 GMAC/s @163 MHz (2.8 @173)	—
Bus job (step 5/6)	249 / 245 clocks per 4×4 job vs 11 compute; 26 MMAC/s	steps 5, 6
Verification	1000 tests, 16,000 results bit-exact vs NumPy; 0 AXI violations	step 6
Bug catching	unsigned mult 997/1000 fail; 17-bit acc 1/1000 (corner only); no skew 6012 errors; short DRAIN 503	steps 2, 4, 6
MNIST network	196 → 32 → 10 (padded 12), INT8; requant M = 40422, S = 24	step 8
MNIST accuracy	97.03% float, **97.00% INT8** (10,000 images); 19/20 in simulation	step 8
Runs per image	104 (416 per batch of 4)	step 8
Step 8 speed	19,000 clk/image, 5,263 img/s @100 MHz, array busy 6.0%	tb_mnist
Step 9a speed	6,520 clk/image, 15,337 img/s, busy 17.5%, **2.9×**	tb_mnist_fast
Bus cost model	write ≈ 5 clocks, read ≈ 4 clocks	solved from the two runs
Where time goes (9a)	writes 72%, STATUS polls 26%, C reads 3%	step 9a
Packing cost	≈ 42 FFs, −6% Fmax	step 9a
UART vs compute	≈ 50 ms vs 110 ns per job (≈ 450,000×)	step 7

Formulas

```
Fmax (MHz)          = 1000 / (T_clk_ns - WNS)            T = 10 ns here
Latency (clocks)    = 3N - 2  (array)   = 3N - 1  (with CLEAR)
Accumulator bits    = 2*DW + ceil(log2 K)                 = 16 + 2 = 18 for K = 4
Skew registers      = N(N-1)/2 per matrix (x DW bits)
Address A[i][j]     = 0x400 + 4*(i*N + j)     C[i][j] = 0xC00 + 4*(i*N + j)
Packed word w       = 0x100 + 4*w  (A),  0x200 + 4*w (B);  byte k = element 4w+k
Requantize          = clamp((max(acc,0)*M + 2^(S-1)) >> S, 0, 127)
Utilization         = (runs x 11) / total clocks
Tiles for C[M x N]  = (K/4) x (N/4) runs per 4 rows of A
```

CHAPTER 15

Interview preparation

15.1 The 60-second pitch

> "I built a matrix-multiply accelerator from scratch: a 4×4 systolic array of INT8 multiply-accumulate units in Verilog. Each processing element maps to one DSP48 slice, and a skew network and a controller FSM feed it, so one 4×4 multiply takes 11 clocks. I wrapped it as an AXI4-Lite peripheral with a proper register map and error responses, and drove it from a PC over a UART-to-AXI bridge I had built in an earlier project.
>
> For verification I used a NumPy golden model with 1000 tests, directed corner cases plus random, and an AXI protocol checker. A corner case caught an accumulator overflow that 991 random tests missed. On the Basys3 I closed timing at 173 MHz by registering the feed path and fixing reset fanout. Then I ran a real INT8 neural network: MNIST at 97% accuracy, bit-exact with Python, using tiling and an accumulate mode.
>
> Measuring that run showed the array was busy only 6% of the time, because the bus was the bottleneck. I packed four INT8 values per bus write, which made the system 2.9× faster for about 40 flip-flops. Next I'm adding double buffering to hide the remaining polling time."

15.2 The 2-minute whiteboard

1. Draw the 4×4 grid: A enters from the left and moves right, B enters from the top and moves down, and each PE keeps C[i][j].
2. Draw the staircase (skew) and explain why row i starts i clocks late.
3. Write the latency: 3N−2, plus 1 for CLEAR = 11.
4. Draw the stack: PC → UART → bridge → AXI master → AXI slave → buffers/FSM → core.
5. Write two numbers: 6% utilization in step 8 and 17.5% after packing. Say "the bus is the bottleneck, and the next step is double buffering."

15.3 Questions and answers

Architecture

Why a systolic array instead of 16 independent MACs reading from memory?

Data reuse and locality. Each operand is fetched once and reused by N PEs as it flows through, and wires are only neighbour-to-neighbour, so bandwidth needs and wire delay stay low as the array grows.

Output-stationary vs weight-stationary?

In mine, the result stays in the PE and A and B both move. In weight-stationary (TPU v1), weights are preloaded into the PEs and partial sums flow out. Output-stationary suits long K (no partial-sum traffic) and makes accumulate-mode tiling trivial. Weight-stationary suits reusing the same weights across many inputs.

What would change for 8×8 or 16×16?

Only the parameter N. The generate loops, skew and FSM all scale; 8×8 passed every test. Latency becomes 3N−1 (23 for 8×8). DSPs go to 64, which still fits the 90 on the Basys3. 16×16 (256 DSPs) needs a bigger FPGA, or LUT multipliers for some PEs.

Peak vs achieved throughput?

Peak is 16 MACs/clock, or 2.6 GMAC/s at 163 MHz. Achieved on MNIST over AXI4-Lite, the array is busy 17.5% of the time. The gap is data movement, which I measured and am closing step by step.

How is this related to NVIDIA tensor cores?

A tensor core is a hardware unit that performs a small matrix multiply-accumulate (D = A×B + C) on tiles each clock. A Volta tensor core does a 4×4×4 FP16 MMA. Bigger matrices are tiled onto it by software, just like my accumulate-mode tiling. The same lessons apply: low-precision inputs with wide accumulators, and keeping the units fed (memory hierarchy, wide loads) matters more than the math itself.

RTL design

Signed multiplication in Verilog: what is the trap?

Both operands must be signed, or the whole expression is evaluated unsigned. −1 × −1 then becomes 65025. That broke 997 of 1000 tests.

How did you choose the accumulator width?

$2{\cdot}DW + \log_2 K$. For K = 4 the worst case is 65536, which needs 18 bits. I used 32 bits to support tiling with K up to about 131,000. A 17-bit version passed every random test and failed only the −128×−128 corner case.

Why is done a level?

Software polls it over a bus and could miss a one-clock pulse. It clears on the next start.

How do you prevent corrupting a run?

The core ignores writes and start while busy, and the AXI slave returns SLVERR for them, so software knows. Tests write mid-run and then check that the result is unchanged.

What is the registered lookahead feed?

The FSM reads column k+1 from the buffer one clock early into a register. The DSP inputs then come from flip-flops instead of counter → RAM → mux logic. Same latency, +21% Fmax.

Why did the reset limit timing, and what does max_fanout do?

One LUT drove reset to about 714 loads, including the DSP reset pins, so the net delay was large. Registering the reset with `max_fanout = 50` lets Vivado replicate the register so each copy drives at most 50 loads. WNS improved from 3.142 to 4.233 ns.

AXI and system

Walk me through one AXI4-Lite write.

The master raises AWVALID with the address and WVALID with data and WSTRB, in any order. My slave latches each into its own holding register when READY is high. When both are present and the B channel is free, it commits the write, then raises BVALID with OKAY, SLVERR or DECERR until BREADY.

What is WSTRB and did you use it?

WSTRB has one bit per byte of WDATA, marking which bytes to write. My packed writes honour it per byte: with WSTRB = 0101, only bytes 0 and 2 are written. tb_packed test 2 checks this.

What is back-pressure and where do you use it?

A receiver holding READY low to slow the sender. My packed-write sequencer needs 4 clocks to unpack, so it stalls the next write until it is done. Without that, a fast master could start the array before the data was written.

How would you feed it faster in a real product?

Double-buffer A and B (load the next tile while computing), use AXI-Stream or a DMA engine for bulk data instead of single-register writes, keep weights resident on chip for reuse, and use an interrupt instead of polling STATUS.

Verification

How do you know it is correct?

Bottom-up unit tests with self-checking testbenches, then a system test against an independent NumPy model (1000 tests, 16,000 results) through the real AXI bus with a protocol checker. Mutation testing: I planted bugs (unsigned multiply, narrow accumulator, no skew, short DRAIN, no stall, ignored WSTRB, swapped bytes) and confirmed each one is caught.

Why an independent golden model?

A Verilog reference written by the same person can share the same misunderstanding as the design. NumPy computes the math independently, and the testbench only moves data and compares.

Tell me about a bug random testing missed.

Two. A 17-bit accumulator failed only the directed −128×−128 test, never the 991 random ones. And removing the bus stall in the packed sequencer was invisible with my own master, which is slower than the sequencer. Only a raw full-speed burst test caught it.

How did you verify the board design without the board?

A board-level testbench drives the UART RX pin bit by bit as the PC would, and checks 10 NumPy jobs end to end through bridge, master, slave and array.

ML and quantization

Explain your quantization.

Symmetric INT8 per tensor: scale = max|W|/127 for weights, inputs mapped 0..127, the hidden scale from the 99.9th percentile of real activations. INT32 accumulation with biases in the accumulator's scale. Requantization uses an integer multiply by M = 40422 and a shift by 24, with rounding and clamping. Everything is integer, so it is bit-exact everywhere.

How did you check the FPGA matches the model?

The testbench compares all 240 output scores for 20 images bit for bit with NumPy: 0 mismatches. Breaking accumulate mode gives 199 mismatches and 18 wrong digits.

Behavioural

What was the hardest part?

Timing closure, because it needed reading the critical path, understanding why it was slow (an unregistered DSP input, high fanout) and fixing the architecture without changing behaviour. Also finding bugs the normal tests could not see.

What would you do differently?

Plan the data movement from day one (packed or stream interfaces, double buffering), because measurement showed the bus dominated. Add functional coverage to the test plan, and keep the project in a path without spaces.

15.4 Resume bullets

- Designed a **4×4 INT8 output-stationary systolic-array accelerator** in Verilog (16 MAC PEs on DSP48E1, skew network, controller FSM); **11-cycle** 4×4 GEMM, parameterised and verified up to 8×8.

- Integrated it as an **AXI4-Lite peripheral** (register map, SLVERR/DECERR, packed WSTRB writes with bus back-pressure), driven from a PC through a custom UART→AXI bridge on a **Basys3 (Artix-7)**.
- Closed timing at **173 MHz (from 125 MHz)** by registering the DSP feed path and replicating a 714-fanout reset; 16 DSP, ≈500 LUT, 0 BRAM.
- Verified against a **NumPy golden model** (1000 tests, 16k results bit-exact, 0 AXI protocol violations) using directed + random tests and mutation testing; a directed corner case caught an overflow 991 random tests missed.
- Ran an **INT8 MNIST MLP (97.0%, within 0.03% of FP32)** on the accelerator, bit-exact to Python, via tiling with a hardware accumulate mode and integer requantization.
- Profiled array utilization (6%) and identified bus overhead as the bottleneck; implemented **packed writes for a 2.9× system speed-up** at the cost of about 42 flip-flops.

CHAPTER 16

What comes next

Step	What	Why it matters
Board	Chapter 12 checklist: program, ping/demo/test, MNIST packed vs unpacked, video	"Did it run on silicon?" becomes "yes, here is the video"
9b	Double buffering: two A/B buffer banks, so the next tile loads while the current one computes; optionally a run queue so CTRL doesn't wait on polling	attacks the 26% polling time; utilization should rise well above 17.5%
10	Scaling and PPA: 4×4 vs 8×8 (and 16×16 estimate): DSP/LUT/FF/Fmax/power table; broadcast (no pass register) vs systolic Fmax; optional hardware requantization	shows you understand area/performance trade-offs, the core of NVIDIA-style hardware interviews
Wrap-up	GitHub repository with README (diagram, results tables, video), final resume bullets, re-rating the project	makes the work visible to recruiters
Future	ASIC flow for the array (you have SCL180 / Cadence experience from your MAC RTL-to-GDS project): synthesis, place and route, area and timing at 180 nm	FPGA + ASIC on the same design is a strong story

APPENDIX A

Register map and software sequence

Address	Name	R/W	Bits
0x000	CTRL	W	bit0 start (self-clearing), bit1 accumulate (C += A×B)
0x004	STATUS	R	bit0 busy, bit1 done
0x008	CYCLES	R	[15:0] clocks of the last run
0x00C	ID	R	0x5A00_0000 | N<<8 | DW = 0x5A000408
0x100–0x10C	A packed	W	word w: bytes = A elements 4w..4w+3 (row w for 4×4)
0x200–0x20C	B packed	W	word w: bytes = B elements 4w..4w+3
0x400–0x43C	A[i][j]	W	[7:0] signed INT8
0x800–0x83C	B[i][j]	W	[7:0] signed INT8
0xC00–0xC3C	C[i][j]	R	[31:0] signed

```
Software for one tile (step 9a):
  for i in 0..3:  write 0x100 + 4i = pack(A row i);  write 0x200 + 4i = pack(B row i)
  write 0x000 = 1        (or 3 to accumulate onto the previous C)
  read  0x004 until (value & 3) == 2
  read  0xC00 + 4*(i*4+j) for the 16 results (only after the last K tile)
```

Full error table

Access	Response
address not a multiple of 4	SLVERR
read A, B, A packed, B packed	SLVERR (write-only)
write STATUS, CYCLES, ID, C	SLVERR (read-only)
write CTRL/A/B/packed while busy	SLVERR
unmapped address (e.g. 0x110, 0x240, 0x010)	DECERR
read CTRL	OKAY, reads 0

APPENDIX B

File map

Files live in the project Library under `systolic-array/stepN/`. The latest version of each file is the one in the highest-numbered step that contains it.

File	Latest in	Vivado set	Role
pe.v	step7 (use_dsp)	Design	processing element
systolic_array.v	step2	Design	N×N grid
skew.v, systolic_core.v	step3	Design	skew + core
systolic_top.v	step8 (accumulate + lookahead)	Design	buffers, FSM, cycles
axil_systolic.v	**step9** (packed)	Design	AXI4-Lite slave
axil_master.v	step5	Design (from step 7)	AXI master (reused)
uart_rx.v, uart_tx.v, uart_axil_bridge.v	step7	Design	PC link (reused)
basys3_systolic.v	step7 (registered reset)	Design, **Top**	board top, LEDs, 7-seg
basys3_systolic.xdc	step7	Constraints	pins, 100 MHz clock
axil_checker.v	step5	Simulation	AXI protocol checker
tb_pe, tb_systolic_array, tb_skew, tb_systolic_core	steps 1–3	Simulation	unit tests
tb_systolic_top.v	step8	Simulation	FSM test (+accumulate port)
tb_axil_systolic.v	step5	Simulation	AXI slave test
tb_system.v + a/b/c_expected/meta.mem	step6	Simulation	1000-test NumPy system test
tb_basys3_systolic.v	step7	Simulation	board test via UART pin
tb_mnist.v + 8 mnist .mem files	step8	Simulation	MNIST, single writes
tb_mnist_fast.v, tb_packed.v	step9	Simulation	MNIST packed; packed directed tests
golden.py	step6	PC	NumPy vectors + check
systolic_uart.py	step7	PC	board ping/demo/test
mnist_model.py, mnist_int8.npz	step8	PC	train + quantize network
mnist_uart.py	**step9**	PC	MNIST on the board (packed by default)

End of guide. Keep this PDF next to the project; when step 9b and step 10 are done, the guide will be updated with their chapters.